

5장 토큰화,
6장 임베딩 (~TF-IDF)

4팀 윤정 조수진 진예원

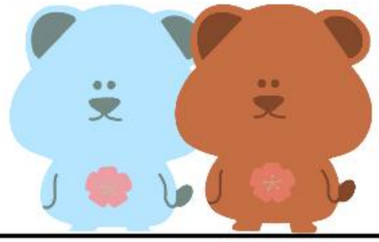
EWHA EURON

목차

#01 토큰화 개요, 단어 및 글자 토큰화, 형태소 토큰화

#02 하위 단어 토큰화

#03 임베딩 개요, 언어 모델, N-gram, TF-IDF



EWHA EURON

토큰화



EWHA EURON

#00 목차

#1 토큰화 개요

#2 단어 및 글자 토큰화

#3 형태소 토큰화

EWHA EURON

#01 토큰화 개요

자연어 : 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어

자연어 처리 (NLP) : 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

목표 : 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하는 것

➡ 해결해야 할 문제들

- 모호성 : 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 됨
- 가변성 : 다양한 사투리, 강세, 신조어, 작문 스타일로 인해 매우 가변적
- 구조 : 문장이나 구의 의미를 이해할 때 구문을 파악하여 의미를 해석

➡ 말뭉치를 일정한 단위인 토큰으로 나누기

EWHA EURON

#01 토큰화 개요

말뭉치 : 자연어 모델을 훈련하고 평가하는데 사용되는 대규모의 자연어
뉴스 기사, 사용자 리뷰, 칼럼 등 목적에 따라 구축되는 텍스트 데이터
언어 모델을 구축하고 평가하는데 자주 사용

토큰 : 개별 단어나 문장 부호와 같은 텍스트
말뭉치보다 작은 단위



토큰화
토크나이저 사용

입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'

결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', 'ㄹ', '수', '있', '다', '.']

EWHA EURON

#01 토큰화 개요

토크나이저 구축 방법

- 공백 분할 : 텍스트를 공백 단위로 분리해 개별 단어로 토큰화
- 정규표현식 적용 : 정규 표현식으로 특정 패턴을 식별해 텍스트 분할
- 어휘 사전 적용 : 사전에 정의된 단어 집합을 토큰으로 사용 → OOV 문제 고려 필요
- 머신러닝 활용 : 데이터세트를 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용

*OOV (Out Of Vocab) : 사전에 없는 단어나 토큰

EWHA EURON

#02 단어 및 글자 토큰화

입력된 텍스트 데이터를 단어나 글자 단위로 나눔 ➡ 각각의 토큰은 의미를 갖는 최소 단위로 분해

- 단어 토큰화
- 글자 토큰화

단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거음은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split()
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거음은', '최고!', '더불어', 'ost는', '더더욱',
'최고!!']
```

띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용

품사 태킹, 개체명 인식, 기계 번역 등 사용

가장 일반적인 토큰화 방법

'최고!' 와 '최고!!', 'cg' 와 'cg.' 은 서로 다른 토큰

➡ 토큰이 단어 사전에 있더라도 OOV가 생김
한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류 등에 취약

EWHA EURON

#02 단어 및 글자 토큰화

글자 토큰화

띄어쓰기 뿐만 아니라 글자 단위로 문장을 나누는 방식

비교적 작은 단어 사전 구축 가능 ➡ 컴퓨터 자원 아낌, 전체 말뭉치 학습 시 각 단어 더 자주 학습 가능

언어 모델링과 같은 시퀀스 예측 작업에 활용 (다음 문자를 예측하는 언어 모델링)

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = list(review)
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시',
'각', '적', ' ', '즐', '거', '음', '은', ' ', '최', '고', '!', ' ', '더', '불', '어', ' ', 'o', 's',
't', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!']
```

영어의 경우 각 알파벳으로 나뉨

한글의 경우 한 글자가 자음과 모음의 조합으로 이루어짐

➡ 자소 단위 토큰화

EWHA EURON

#02 단어 및 글자 토큰화

자소 단위 토큰화 – 자모 라이브러리

한글 문자 및 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리

텍스트를 자소 단위로 분해해 토큰화를 수행

```
retval = jamo.h2j(
    hangul_string
)
```

자모 변환 함수

입력된 한글 문자열을 유니코드 U+1100 ~ U+11FE 사이의 조합형 한글 자모로 변환

- 조합형 : 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글로 표현
- 완성형 : 조합된 글자 자체에 값을 부여해 인코딩하는 방식

h2j 함수

```
retval = jamo.j2hcj(
    jamo
)
```

한글 호환성 자모 변환 함수

조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수

초성, 중성, 종성으로 구성 → 완성형 한글을 쉽게 나눌 수 있음

EWHA EURON

#02 단어 및 글자 토큰화

자모 변환 함수와 한글 호환성 자모 변환 함수 활용

➡ 자소 단위로 분리

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거음은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

```
['ㅎ', 'ㅕ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅘ', ' ', 'ㄱ', 'ㅜ', 'ㅂ', 'ㅜ', 'ㄴ', ' ', 'ㅂ', 'ㅜ',
'ㄹ', 'ㄱ', 'ㅏ', 'ㄴ', 'ㅡ', 'ㅇ', 'ㅎ', 'ㅏ', 'ㄴ', ' ', 'c', 'g', '.', ' ', 'ㅅ', 'ㅣ', 'ㄱ',
'ㅏ', 'ㄱ', 'ㅈ', 'ㅓ', 'ㄱ', ' ', 'ㅈ', 'ㅡ', 'ㄹ', 'ㄱ', 'ㅓ', 'ㅇ', 'ㅡ', 'ㅁ', 'ㅇ', 'ㅡ', 'ㄴ',
' ', 'ㅊ', 'ㅚ', 'ㄱ', 'ㅗ', '!', ' ', 'ㄷ', 'ㅓ', 'ㅂ', 'ㅜ', 'ㄹ', 'ㅇ', 'ㅓ', ' ', 'o', 's',
't', 'ㄴ', 'ㅡ', 'ㄴ', ' ', 'ㄷ', 'ㅓ', 'ㄷ', 'ㅓ', 'ㅇ', 'ㅜ', 'ㄱ', ' ', 'ㅊ', 'ㅚ', 'ㄱ', 'ㅗ',
'!', '!']
```

- 단어 토큰화에 비해 비교적 적은 크기의 단어 사전 구축 가능
- 접사와 문장 부호의 의미 학습 가능 (OOV 감소)
- 개별 토큰이 의미가 없음
  - ➡ 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야함
- 다의어, 동음이의어가 많은 도메인에서 구별 어려움
- 모델 입력 시퀀스가 길어질수록 연산량 증가

EWHA EURON

#03 형태소 토큰화

텍스트를 형태소 단위로 나누는 방법

언어의 문법과 구조를 고려해 단어를 분리하고 의미 있는 단위로 분류

다양한 접사와 조사가 조합되어 하나의 낱말을 이루는 한국어와 같은 교착어에서 중요하게 수행

형태소 : 실제로 의미를 가지고 있는 최소 단위

- 자립 형태소 : 스스로 의미를 가짐
  단어의 기본이 되는 형태소
  명사, 동사, 형용사와 같은 단어를 이루는 기본 단위
  예시 - ‘그’, ‘나’, ‘인사’
- 의존 형태소 : 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용
  자립 형태소와 함께 조합되어 특정 역할 수행
  조사, 어미, 접두사, 접미사에 해당
  예시 - ‘-는’, ‘-에게’, ‘-를’

EWHA EURON

#03 형태소 토큰화

형태소 어휘 사전 : 자연어 처리에서 사용되는 어휘 사전 중 각 단어의 형태소 정보를 포함하는 사전

각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 등의 정보 포함

형태소 분석 작업에서 중요한 역할

품사 태깅 : 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업

문맥을 고려하여 더욱 정확한 분석 가능

➡ KoNLPy, NLTK, spaCy 라이브러리 활용

EWHA EURON

#03 형태소 토큰화

KoNLPy

한국어 자연어 처리를 위해 개발된 라이브러리

명사 추출, 형태소 분석, 품사 태깅 등 기능 제공

텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에서 사용

Okt(Open Korean Tex), 꼬꼬마(Kkma), 한나눔(Hannanum) 등 다양한 형태소 분석기 지원

EWHA EURON

#03 형태소 토큰화

Okt 토큰화

```
from konlpy.tag import Okt

okt = Okt()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
```

```
pos = okt.pos(sentence)

print("명사 추출 :", nouns)
print("구 추출 :", phrases)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
구 추출 : ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 추출 : ['무엇', '이든', '상상', '할', '수', '있는', '사람', '은', '무엇', '이든', '만들어',
'낼', '수', '있다', '.']
품사 태깅 : [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'),
('있는', 'Adjective'), ('사람', 'Noun'), ('은', 'Josa'), ('무엇', 'Noun'), ('이든', 'Josa'),
('만들어', 'Verb'), ('낼', 'Noun'), ('수', 'Noun'), ('있다', 'Adjective'), ('.', 'Punctuation')]
```

문장을 입력 받아 명사, 구, 형태소, 품사 등의 정보를 추출하는 여러가지 메소드 제공

명사 추출, 구문 추출, 형태소 추출, 품사 태깅이 대표적

- 입력된 문장에서 각 명사, 어절 구 단위, 형태소만 추출해 리스트로 반환
- 각 단어에 대한 품사 정보를 추출하여 (형태소, 품사) 형태의 튜플로 구성된 리스트로 반환

EWHA EURON

#03 형태소 토큰화

꼬꼬마 토큰화

```
from konlpy.tag import Kkma

kkma = Kkma()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."
```

```
nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)

print("명사 추출 :", nouns)
print("문장 추출 :", sentences)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇']
구문 추출 : ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
형태소 추출 : ['무엇', '이', '든', '상상', '하', 'ㄹ', '수', '있', '는', '사람', '은', '무엇', '이',
'든', '만들', '어', '내', 'ㄹ', '수', '있', '다', '.']
품사 태깅 : [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄹ',
'ETD'), ('수', 'NNB'), ('있', 'VV'), ('는', 'ETD'), ('사람', 'NNG'), ('은', 'JX'), ('무엇', 'NP'),
('이', 'VCP'), ('든', 'ECE'), ('만들', 'VV'), ('어', 'ECD'), ('내', 'VXV'), ('ㄹ', 'ETD'), ('수',
'NNB'), ('있', 'VV'), ('다', 'EFN'), ('.', 'SF')]
```

명사, 문장, 형태소, 품사 추출 가능

구문 추출 대신 명사 추출 중 kkma.sentences 기능 제공

EWHA EURON

#03 형태소 토큰화

Okt 토큰화

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
구 추출 : ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 추출 : ['무엇', '이든', '상상', '할', '수', '있는', '사람', '은', '무엇', '이든', '만들어',
'낼', '수', '있다', '.']
품사 태깅 : [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'),
('있는', 'Adjective'), ('사람', 'Noun'), ('은', 'Josa'), ('무엇', 'Noun'), ('이든', 'Josa'),
('만들어', 'Verb'), ('낼', 'Noun'), ('수', 'Noun'), ('있다', 'Adjective'), ('.', 'Punctuation')]
```

동일한 메소드여도 형태소 분석기의 특징에 따라 다른 결과가 나타남

시스템의 목적과 환경에 맞는 적절한 형태소 분석기 사용

➡ 19개

꼬꼬마 토큰화

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇']
구문 추출 : ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
형태소 추출 : ['무엇', '이', '든', '상상', '하', 'ㄹ', '수', '있', '는', '사람', '은', '무엇', '이',
'든', '만들', '어', '내', 'ㄹ', '수', '있', '다', '.']
품사 태깅 : [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄹ',
'ETD'), ('수', 'NNB'), ('있', 'VV'), ('는', 'ETD'), ('사람', 'NNG'), ('은', 'JX'), ('무엇', 'NP'),
('이', 'VCP'), ('든', 'ECE'), ('만들', 'VV'), ('어', 'ECD'), ('내', 'VXV'), ('ㄹ', 'ETD'), ('수',
'NNB'), ('있', 'VV'), ('다', 'EFN'), ('.', 'SF')]
```

➡ 56개

EWHA EURON

#03 형태소 토큰화

Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 콜론, 빗금
JX	보조사	SO	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

EWHA EURON

#03 형태소 토큰화

NLTK

토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등 기능 제공

영어 자연어 처리를 위해 개발, 다양한 언어를 위한 데이터와 모델도 제공

Punkt 모델과 Averaged Perceptron Tagger 모델 활용

- 모두 대규모 영어 말뭉치 기반 트리 뱅크로 학습

EWHA EURON

#03 형태소 토큰화

Punkt 모델 기반 영문 토큰화

```
from nltk import tokenize

sentence = "Those who can imagine anything, can create the impossible."

word_tokens = tokenize.word_tokenize(sentence)
sent_tokens = tokenize.sent_tokenize(sentence)

print(word_tokens)
print(sent_tokens)
```

```
['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible', '.']
['Those who can imagine anything, can create the impossible.']
```

word_tokenize : 공백을 기준으로 단어 분리, 구두점 등을 처리해 각각의 단어를 추출

sent_tokenize : 마침표, 느낌표, 물음표 등의 구두점을 기준으로 문장을 분리

EWHA EURON

#03 형태소 토큰화

Averaged Perceptron Tagger 모델 기반 영문 품사 태깅

```
from nltk import tag
from nltk import tokenize

sentence = "Those who can imagine anything, can create the impossible."

word_tokens = tokenize.word_tokenize(sentence)
pos = tag.pos_tag(word_tokens)

print(pos)
```

```
[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), (',', ','),
('can', 'MD'), ('create', 'VB'), ('the', 'DT'), ('impossible', 'JJ'), ('.', '.')]
```

토큰화된 단어된 문장에서 품사 태깅 수행

Average Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP$	소유격 대명사
FW	외래어	RB	부사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP$	소유격 관계대명사
UH	감탄사	WRB	관계부사

EWHA EURON

#03 형태소 토큰화

spaCy

Cython 기반으로 개발된 오픈소스 라이브러리

NLTK와 다르게 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반 라이브러리

NLTK보다 모델이 더 크고 복잡하며 더 많은 리소스 요구

24개 이상 언어로 사전 학습된 모델을 제공

EWHA EURON

#03 형태소 토큰화

```
import spacy
  영어로 사전 학습된 모델 불러오기
nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

객체 지향적으로 구현

➡ 처리한 결과를 doc 객체에 저장

➡ doc의 token 객체에 대한 정보 기반으로 처리

기본 품사 속성, 원본 텍스트 데이터, 벡터, 벡터 노름, 세분화 품사 속성 등의 속성 포함

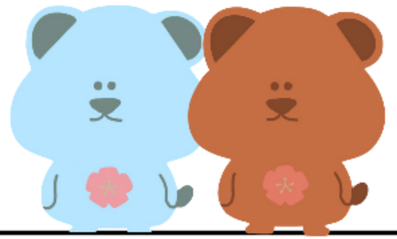
```
[PRON  - DT ] : Those
[PRON  - WP ] : who
[AUX   - MD ] : can
[VERB  - VB ] : imagine
[PRON  - NN ] : anything
[PUNCT - ,  ] : ,
[AUX   - MD ] : can
[VERB  - VB ] : create
[DET   - DT ] : the
[ADJ   - JJ ] : impossible
[PUNCT - .  ] : .
```

spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	…	생략 부호
CCONJ	CC	접속사	PUNCT	“	여는 따옴표
DET	DT	관사, 한정사	PUNCT	”	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	$	통화
DET	WP$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

THANK YOU



EWHA EURON

하위 단어 토큰화



Subword Tokenization · 5.3

EWHA EURON

#00 목차

#01 **하위 단어 토큰화가 필요한 이유**
형태소 분석기의 한계 , 하위 단어 토큰화란 ?

#02 **바이트 페어 인코딩 (BPE)**
가장 자주 붙어 나오는 쌍을 병합

#03 **실습 ① SentencePiece 로 BPE 학습**
예제 5.10 ~ 5.14 · 청와대 청원 말뭉치

#04 **워드피스 (WordPiece)**
빈도 대신 점수로 병합

#05 **실습 ② Tokenizers 로 WordPiece 학습**
예제 5.15 ~ 5.16 · 정규화 → 사전 토큰화 → 모델 → 디코더

#06 **그 외 알고리즘과 총정리**
BBPE, Unigram, 토큰화 방법 비교

EWHA EURON

#01 하위 단어 토큰화가 필요한 이유

형태소 토큰화는 한국어에 효과적이다. 하지만 언어는 계속 변한다.
새로운 단어와 표현이 생기고, 디지털 매체의 글은 맞춤법 · 띄어쓰기가 엄격하지 않고 오탈자가 많다.

실패 사례 ① 외래어와 오탈자

원문	형태소 분석 결과	무엇이 문제인가
시보리도 짱짱해고 허리도 어벙하지안구 죠하효	['시', '보리', '도', '짱짱해고', '허리', '도', '어', '벙하지안구', '죠', '하', '효']	**외래어 '시보리'**를 시 + 보리로 잘못 쪼갬 **'어벙하다'**를 어 + 벙하지안구로 엉뚱하게 나눔

실패 사례 ② 신조어

원문	형태소 분석 결과	바람직한 분해
진짜 멋있네요! 돈쭐날만 하네요.	['진짜', '멋있네요', '!', '돈쭐날', '만', '하네요', '.']	돈쭐 + 날 + 만 또는 돈쭐 + 날만

*'돈쭐내다' = '돈' + '혼쭐내다'를 합친 신조어 (착한 가게의 물건을 팔아주자는 뜻) *OOV (Out Of Vocabulary) : 어휘 사전에 없는 단어나 토큰

핵심 문제 분석기가 '돈쭐-'을 모르니까 돈쭐내러, 돈쭐나, 돈쭐내다가 전부 따로 사전에 들어간다 → 어휘 사전이 커지고 OOV 에 대응하기 어렵다.
형태소 분석기는 모르는 단어를 적절한 단위로 나누는 것에 약하다 (전문용어 · 고유어가 많은 데이터에서 특히).

EWHA EURON

#01 하위 단어 토큰화란 ?

정의 : 하나의 단어를 자주 쓰이는 하위 단어 (Subword) 들의 조합으로 나누는 토큰화 방법



효과

- 단어 길이 (토큰 단위) 를 줄여서 처리 속도가 빨라진다
- OOV, 신조어 , 은어 , 고유어 문제를 완화한다
  - 처음 보는 단어도 아는 조각들로 조립할 수 있기 때문

레고로 비유하면

단어 토큰화
완성품 블록만 파는 가게

글자 토큰화
1 × 1 블록만 파는 가게

하위 단어 토큰화
자주 쓰는 모양은 큰 블록 , 드문 모양은 작은 블록
→ 처음 보는 작품 (신조어) 도 아는 블록으로 조립

종류

바이트 페어 인코딩 (BPE) | **워드피스 (WordPiece)** | 유니그램 (Unigram)

← 책에서는 BPE 와 WordPiece 를 다룬다

최근 동향 하위 단어 토큰화가 표준 – BERT 는 WordPiece, GPT 계열은 BBPE(바이트 단위 BPE)

EWHA EURON

#02 바이트 페어 인코딩 (BPE)

개념

- 다이그램 코딩 (Digram Coding) 이라고도 부른다
- 원래는 데이터 압축 알고리즘이다
  - 가장 자주 등장하는 글자 쌍을 찾아 새 기호로 바꾼다
- NLP 에서는 하위 단어 토큰화 방법으로 쓰인다 (Sennrich et al., 2016)
- 결과 : 자주 나오는 단어는 한 토큰이 되고, 드문 단어는 여러 토큰의 조합으로 표현된다

언제 멈추나 ?

① 더 이상 병합할 쌍이 없을 때 또는 ② 정해둔 어휘 사전 크기에 도달했을 때

→ 사전 크기를 정해 두면 " 병합을 몇 번 할지 " 가 정해진다 (실습의 vocab_size)

EWHA EURON

#02 바이트 페어 인코딩 (BPE)

압축 알고리즘으로서의 BPE — abracadabra

단계	결과	무엇을 바꿨나
원문	abracadabra	—
Step 1	AracadAra	가장 많이 나온 쌍 ab (2 번) 를 새 글자 A 로 치환
Step 2	ABcadAB	ra (2 번) 를 B 로 치환
Step 3	CcadC	치환한 글자끼리의 쌍 AB 도 대상이 됨 → C 로 치환
끝	CcadC	더 이상 2 번 이상 나오는 쌍이 없음 → 압축 종료

- 치환에 쓰는 새 글자는 입력에 없는 글자라면 아무거나 써도 된다
- 11 글자가 5 글자로 줄었다

주의 Step 1 에서 ab, br, ra 가 모두 2 번으로 동점이다. 책은 ab 를 골랐을 뿐이고, 동점일 때 무엇을 고를지는 구현마다 다르다.

EWHA EURON

#02 토크나이저로서의 BPE

압축과의 차이 : 글자를 치환해서 지우는 대신 , 병합한 쌍을 어휘 사전에 **" 추가 "** 한다 → 사전이 조각에서 큰 조각 순서로 자라난다

예시 말뭉치

빈도 사전

('low', 5), ('lower', 2),
('newest', 6), ('widest', 3)

① 초기화 — 글자로 쪼갠다

빈도 사전

('l','o','w', 5), ('l','o','w','e','r', 2),
('n','e','w','e','s','t', 6),
('w','i','d','e','s','t', 3)

어휘 사전 (10 개)

['d','e','i','l','n','o','r','s','t','w']

② 인접한 글자 쌍의 빈도를 센다

쌍	어디서 나오나	빈도
e-s	**newest(6) + widest(3)**	**9 ← 최다**
s-t	newest(6) + widest(3)	9
w-e	lower(2) + newest(6)	8
l-o, o-w	low(5) + lower(2)	7
n-e, e-w	newest(6)	6
w-i, i-d, d-e	widest(3)	3
e-r	lower(2)	2

→ 쌍이 나올 때마다 그 단어의 빈도만큼 더한다 . e-s 는 newest(6) + widest(3) = 9 로 최다 (s-t 와 동점) → e-s 부터 병합

EWHA EURON

#02 바이트 페어 인코딩 (BPE)

③ 최다 쌍을 병합하고 어휘 사전에 추가한다 → 반복

병합 회차	병합한 쌍 → 새 토큰	빈도	병합 후 빈도 사전 (변한 부분)
1	e + s → es	9	('n','e','w','es','t', 6), ('w','i','d','es','t', 3)
2	es + t → est	9	('n','e','w','est', 6), ('w','i','d','est', 3)
3	l + o → lo	7	('lo','w', 5), ('lo','w','e','r', 2)
4	lo + w → low	7	('low', 5), ('low','e','r', 2)
5 ~ 7	ne → new → newest	6	('newest', 6)
8 ~ 10	wi → wid → widest	3	('widest', 3)

직관 BPE 는 " 가장 자주 붙어 다니는 단짝부터 한 팀으로 묶기 " 다. 묶인 팀도 다시 다른 팀과 묶일 수 있다 (es → est).

EWHA EURON

#02 바이트 페어 인코딩 (BPE)

④ 10 번 병합한 결과

빈도 사전	('low', 5), ('low','e','r', 2), ('newest', 6), ('widest', 3)
어휘 사전 (10 + 10 = 20 개)	['d','e','i','l','n','o','r','s','t','w', **'es','est','lo','low','ne','new','newest','wi','wid','widest']**

⑤ 처음 보는 단어 (OOV) 도 처리된다

새 단어 (말뭉치에 없음)	BPE 토큰화 결과
newer	new + e + r
wider	wid + e + r
lowest	low + est

단어 사전이었다면 셋 다 OOV 다 .
BPE 는 아는 조각으로 조립해서 처리한다 .
최악의 경우에도 글자 단위까지 내려가면 된다 .

참고 Sennrich 논문에서는 단어 끝에 </w> 표시를 붙여서 newest 의 est(단어 끝) 와 estimate 의 est(단어 시작) 를 구별한다 . 책은 이 부분을 생략했다 .

EWHA EURON

#03 실습 ① SentencePiece 로 BPE 학습

사용하는 라이브러리

라이브러리	만든 곳	역할
SentencePiece (센텐스피스)	구글	오픈소스 하위 단어 토크나이저 . BPE 와 유사한 알고리즘으로 토큰화하고 단어 사전을 만든다 . 세밀한 하이퍼파라미터 설정이 가능하다
Korpora (코포라)	ko-nlp (오픈소스)	국립국어원 , AI Hub 등의 한국어 말뭉치를 파이썬 API 로 쉽게 내려받는다 . 말뭉치 목록은 Korpora.corpus_list() 로 확인

설치 `pip install sentencepiece Korpora`

전체 흐름

예제 5.10 말뭉치 불러오기 → **예제 5.11** corpus.txt 로 저장 → **예제 5.12** 토크나이저 학습 → **예제 5.13** 토큰화 · 인코딩 · 디코딩 → **예제 5.14** 어휘 사전 확인

학습 결과 : petition_bpe.model + petition_bpe.vocab 생성

EWHA EURON

#03 예제 5.10 — 청와대 청원 데이터 불러오기

코드

```
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]

print("청원 시작일 :", petition.begin)
print("청원 종료일 :", petition.end)
print("청원 동의 수 :", petition.num_agree)
print("청원 범주 :", petition.category)
print("청원 제목 :", petition.title)
print("청원 본문 :", petition.text[:30])
```

출력

```
청원 시작일 : 2017-08-25
청원 종료일 : 2017-09-24
청원 동의 수 : 88
청원 범주 : 육아/교육
청원 제목 : 학교는 인력센터, 취업센터가 아닙니다. ...
```

코드	의미
Korpora.load("korean_petitions")	이름만 넣으면 말뭉치를 내려받아 객체로 돌려준다
corpus.train	말뭉치 중 학습용 데이터
dataset[0]	첫 번째 청원 하나
.begin / .end	청원 시작일 / 종료일
.num_agree	동의 수
.category	범주
.title / .text[:30]	제목 / 본문 앞 30 글자만

데이터 2017 년 8 월 ~ 2019 년 3 월 청와대 국민청원 게시판의 청원 , 총 433,631 개 .
왜 이 데이터 ? 실제 사람이 쓴 구어체 글이라 오탈자 · 신조어 · 띄어쓰기 오류가 많다 → 형태소 분석기의 약점이 드러나는 데이터라 하위 단어 토큰화를 시험하기 좋다

EWHA EURON

#03 예제 5.11 — 학습용 텍스트 파일 만들기

코드

```
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
with open("../datasets/corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition + "\n")
```

코드	의미
corpus.get_all_texts()	말뭉치의 청원 본문 전체를 한 번에 가져온다
open("../datasets/corpus.txt", "w", encoding="utf-8")	corpus.txt 파일을 쓰기 모드 ("w") 로 연다 . utf-8 은 한글이 깨지지 않게 하는 설정
for petition in petitions:	청원을 하나씩 꺼낸다
f.write(petition + "\n")	청원 하나를 한 줄로 쓰고 줄을 바꾼다 → 청원 하나당 한 줄

왜 파일로 저장할까 ? SentencePiece 는 텍스트 파일 경로를 입력으로 받는다 . 그래서 먼저 말뭉치를 corpus.txt 하나로 저장한다 .

EWHA EURON

#03 예제 5.12 — 토크나이저 모델 학습

코드

```
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=../datasets/corpus.txt\
    --model_prefix=petition_bpe\
    --vocab_size=8000 model_type=bpe"
)
```

- SentencePieceTrainer.Train(...) : 토크나이저를 학습하는 메서드
- 인자를 하나의 문자열로 넘긴다
  - 명령줄 옵션처럼 -- 이름 = 값 을 띄어쓰기로 이어 붙인다
- --input : 학습할 말뭉치 파일 (5.11 에서 만든 corpus.txt)
- --model_prefix : 만들어질 모델 파일의 이름
- --vocab_size=8000 : 어휘 사전 크기
- model_type=bpe : BPE 알고리즘 사용
- 줄 끝의 \ 는 문자열을 다음 줄로 이어 쓰기 위한 표시 → 결과적으로 한 줄짜리 문자열

학습 결과 petition_bpe.model (학습된 토크나이저) 과 petition_bpe.vocab (어휘 사전) 두 파일이 생긴다

EWHA EURON

#03 표 5.5 SentencePieceTrainer 주요 매개변수

매개변수	의미	이 예제의 값 / 기본값
input	말뭉치 텍스트 파일 경로	../datasets/corpus.txt
model_prefix	만들어질 모델 파일의 이름	petition_bpe → petition_bpe.model, .vocab
vocab_size	어휘 사전 크기 (= BPE 를 언제 멈출지)	8000
character_coverage	말뭉치의 글자 중 토크나이저가 다룰 글자의 비율	기본 0.9995 (한국어 · 중국어처럼 글자 종류가 많은 언어용). 영어처럼 글자 수가 적으면 1.0
model_type	토크나이저 알고리즘	unigram(기본값), bpe, char, word → 이 예제는 bpe
max_sentence_length	최대 문장 길이	–
unk_id	OOV 를 뜻하는 <unk> 토큰의 ID	기본 0
bos_id	문장 시작 <s> 토큰의 ID (Beginning Of Sentence)	기본 1
eos_id	문장 끝 </s> 토큰의 ID (End Of Sentence)	기본 2

vocab_size 가 왜 중요할까 ? BPE 는 " 병합을 몇 번 할지 " 가 곧 사전 크기다. 크면 단어 통째 토큰이 많아지고 (시퀀스는 짧음), 작으면 조각 토큰이 많아진다 (시퀀스는 김). 사전 크기 vs 시퀀스 길이의 줄다리기를 이 숫자 하나로 조절한다.

EWHA EURON

#03 학습 결과와 ▁ 표시

학습 결과로 생기는 파일

파일	내용
petition_bpe.model	학습된 토크나이저 자체 (불러와서 사용)
petition_bpe.vocab	어휘 사전 (토큰과 점수 목록)

▁ (U+2581) — SentencePiece 의 공백 표시법

SentencePiece 는 띄어쓰기도 하나의 문자처럼 다룬다 . 공백을 특수문자 ▁로 바꿔서 토큰 안에 넣는다 .



- ▁가 붙은 토큰은 " 여기서 새 단어가 시작된다 " 는 뜻이다
- 공백 정보가 토큰에 남아 있어서 , 디코딩할 때 원문 띄어쓰기를 그대로 복원할 수 있다
  - 띄어쓰기 규칙이 언어마다 달라도 같은 방식으로 처리할 수 있다 (언어 독립적)

*▁ 는 일반 밑줄 _ 과 다른 글자다 . 모양만 비슷하다 .

EWHA EURON

#03 예제 5.13 — BPE 토큰화하기

코드

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니다"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)
encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(tokenized_sentences)
```

코드	의미
SentencePieceProcessor()	토크나이저 객체 생성
tokenizer.load("petition_bpe.model")	5.12 에서 학습한 모델 불러오기
sentence / sentences	문장 하나 (문자열) 와 여러 문장 (리스트) 을 준비
encode_as_pieces	문장 → 하위 단어 리스트
encode_as_ids	문장 → 정수 ID 리스트
decode_ids	정수 ID → 문장
decode_pieces	하위 단어 → 문장

포인트 토큰화 · 인코딩 · 디코딩 메서드 모두 문장 하나 (문자열) 나 여러 문장 (리스트) 을 똑같이 넣을 수 있다

EWHA EURON

#03 예제 5.13 — 출력 결과와 해석

출력 (위에서부터 단일 문장 토큰화 / 여러 문장 토큰화 / 단일 문장 정수 인코딩 / 디코딩)

```
['_안녕하세요', ',', '_토', '크', '나', '이', '저', '가', '_잘', '_학', '습', '되었', '군요', '!']
[['_이렇게', '_입', '력', '값을', '_리', '스트', '로', '_받아서'],
 ['_쉽게', '_토', '크', '나', '이', '저', '를', '_사용할', '_수', '_있', '답니다']]
[664, 6553, 991, 6880, 6544, 6513, 6590, 6523, 159, 110, 6554, 868, 782, 6648]
['이렇게 입력값을 리스트로 받아서', '쉽게 토크나이저를 사용할 수 있답니다']
```

→ **디코딩하면 띄어쓰기까지 원문과 똑같이 복원된다**

청원에서 자주 쓰는 말 → 통째로 한 토큰

_ 안녕하세요 | _ 이렇게 | _ 받아서

'토크나이저'는 청원에서 거의 안 쓰는 말 → 잘게 쪼개짐

_ 토 | 크 | 나 | 이 | 저

→ **그래도 <unk>(OOV) 가 되지는 않았다!** 자주 나오는 말은 한 덩어리로, 드문 말은 아는 조각으로 조립하는 하위 단어 토큰화의 장점이 그대로 보인다.
같은 단어라도 어떤 데이터로 학습했느냐에 따라 쪼개지는 방식이 달라진다.

EWHA EURON

#03 예제 5.14 — 어휘 사전 확인

코드

```
vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
print(list(vocab.items())[:5])
print("vocab size :", len(vocab))
```

출력

```
[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '▁이')]
vocab size : 8000
```

코드	의미
get_piece_size()	어휘 사전의 토큰 개수 (= 8000)
id_to_piece(idx)	정수 ID → 하위 단어 (반대는 piece_to_id)
{idx: ... for idx in range(...)}	딕셔너리 컴프리헨션으로 {ID: 토큰} 사전을 만든다
list(vocab.items())[:5]	앞의 5 개만 출력

특수 토큰 3 개가 앞번호를 차지한다

ID	토큰	의미
0	<unk>	unknown. OOV 는 이 토큰으로 매핑
1	<s>	문장 시작
2	</s>	문장 끝

그 다음부터 '니다', '▁이' 처럼 청원 말뭉치에서 자주 나온 조각이 이어진다. 존댓말 어미 '니다' 가 상위권인 것이 청원 글다운 특징!

#04 워드피스 (WordPiece)

- BPE 와 거의 같은 방식으로 학습한다 (글자로 쪼갠 뒤 쌍을 하나씩 병합하는 과정을 반복)
- 차이 : 어떤 쌍을 병합할지 고르는 기준이 빈도가 아니라 확률 (점수) 이다
- 새 하위 단어를 만들 때 그 쌍이 함께 나타날 확률을 계산해 가장 확률이 높은 쌍을 고른다
  - 이렇게 선택된 하위 단어는 이후에도 높은 확률로 선택되기 쉬워서 , 더 정확한 하위 단어로 나눌 수 있다
- BERT 가 사용하는 토크나이저로 유명하다 . 종료 조건은 BPE 와 같다

	BPE	WordPiece
병합 기준	쌍의 빈도 f(x, y)	점수 f(x, y) ÷ (f(x) · f(y))
같은 예시의 1 회차	e + s (9 번 , 최다)	i + d (점수 0.33)
성향	흔한 조합부터 묶음	서로 강하게 결합된 조합부터 묶음
하위 단어 표시	단어 시작에 ▁ (SentencePiece)	단어가 이어지는 부분에 ##
대표 사용 모델	GPT 계열 (BBPE 변형), 많은 번역 모델	BERT

*WordPiece 점수는 NLP 수업에서 배운 PMI(점별 상호정보량) 와 같은 발상 : " 따로 나올 확률에 비해 같이 나올 확률이 얼마나 큰가 "

EWHA EURON

#04 병합 점수 공식 (수식 5.1)

$$score = \frac{f(x, y)}{f(x) \cdot f(y)}$$

* 교재 수식의 쉼표 (f(x), f(y)) 는 곱셈 기호 (·) 를 뜻한다

기호	뜻
f	빈도 (frequency). 몇 번 나왔는지 세는 함수
x, y	병합하려는 두 하위 단어 (예 : i 와 d)
f(x, y)	x 바로 뒤에 y 가 붙어서 나온 횟수 = 쌍 xy 의 빈도
f(x), f(y)	x 혼자의 빈도 , y 혼자의 빈도
f(x) · f(y)	두 빈도를 곱한 값 (가운데 점은 곱셈)
score	x 와 y 를 합치는 것이 적절한지 판단하는 점수 . 클수록 병합

어떻게 읽을까 ?

- 분자는 " 같이 나온 횟수 ", 분모는 " 각자 얼마나 흔한지 "
- 흔한 글자끼리는 우연히 붙어 나와도 분모가 커서 점수가 낮다
- 드물어도 나올 때마다 꼭 같이 나오는 짝이면 점수가 높다

BPE 자주 붙어 다니는 커플
WordPiece 서로만 만나는 커플
을 먼저 묶는다

EWHA EURON

#04 같은 예시로 계산해보기

초기 상태 (BPE 와 똑같이 글자로 쪼갠다)

빈도 사전	('l','o','w', 5), ('l','o','w','e','r', 2), ('n','e','w','e','s','t', 6), ('w','i','d','e','s','t', 3)
각 글자의 빈도	l 7, o 7, w 16, e 17, r 2, n 6, s 9, t 9, i 3, d 3

1 회차 점수 비교

쌍	쌍 빈도 f(x, y)	계산	점수
e-s (BPE 라면 1 순위)	9	9 ÷ (17 × 9)	≈ 0.06
s-t	9	9 ÷ (9 × 9)	≈ 0.11
l-o	7	7 ÷ (7 × 7)	≈ 0.14
i-d	**3**	**3 ÷ (3 × 3)**	**≈ 0.33 ← 최고**

빈도는 3 밖에 안 되지만
i 와 d 는 나올 때마다 붙어 나온다

→ i + d → id 로 병합

2 회차

l-o 쌍은 7 번 , l 과 o 도 각각 7 번 → 7 ÷ (7 × 7) ≈ 0.14 로 최고 → **lo** 로 병합
어휘 사전 : ['d','e','i','l','n','o','r','s','t','w', 'id', 'lo']

EWHA EURON

#05 실습 ② Tokenizers 로 WordPiece 학습

허깅페이스 (Hugging Face) 의 Tokenizers 라이브러리 – WordPiece API 로 토크나이저를 쉽고 빠르게 구현 · 학습할 수 있다

Tokenizers 의 처리 파이프라인

설치 `pip install tokenizers`



단계	하는 일	예시
① 정규화	텍스트를 일관된 형식으로 표준화. 모호한 경우를 막으려고 일부 문자를 바꾸거나 지운다	불필요한 공백 제거, 소문자 변환, 유니코드 정규화, 구두점 · 특수문자 처리
② 사전 토큰화	본격적인 토큰화 전에 단어처럼 큰 단위로 먼저 나눈다	공백이나 구두점 기준으로 분리
③ 모델	나눠진 단어 안에서 하위 단어로 쪼갠다	WordPiece
④ 디코더	토큰을 다시 문장으로 붙인다	## 떼고 이어 붙이기

* 유니코드 정규화가 필요한 이유 : 같은 글자도 여러 유니코드로 표현될 수 있다. "가" 는 완성형 코드 하나로도, ㄱ + ㅏ 두 코드로도 쓸 수 있어서 한 형태로 통일해야 같은 토큰으로 인식한다

EWHA EURON

#05 예제 5.15 — WordPiece 토크나이저 학습

코드

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()

tokenizer.train(["../datasets/corpus.txt"])
tokenizer.save("../models/petition_wordpiece.json")
```

코드	의미
Tokenizer(WordPiece())	③ WordPiece 알고리즘을 쓰는 빈 토크나이저를 만든다
Sequence([NFD(), Lowercase()])	① 정규화 여러 개를 순서대로 적용 : NFD 유니코드 정규화 → 소문자 변환
Whitespace()	② 사전 토큰화 : 공백과 구두점 기준으로 먼저 자른다
train([파일 경로])	파일 목록을 받아 학습. 리스트라서 파일 여러 개도 가능 (BPE 때와 같은 청원 말뭉치)
save(경로)	학습 결과를 JSON 파일 하나로 저장. 어휘 사전 + 정규화 · 사전 토큰화 설정(메타데이터)이 함께 들어간다

EWHA EURON

#05 예제 5.15 — 실행할 때 주의할 점

NFD 와 한글 NFD 는 한글 음절을 초성 · 중성 · 종성 자모로 분해한다 ("가" → ㄱ + ㅏ). 파트 A 에서 본 jamo 의 h2j() 와 같은 효과다. 화면에는 합쳐진 글자로 보이지만 토큰 내부는 자모 단위다. 실제로 '안녕하세요' 토큰의 길이를 재보면 5 가 아니라 **12** 다 (직접 확인함).

실행 오류 책처럼 WordPiece() 를 기본값으로 만들면 어휘 사전에 [UNK] 토큰이 없다. 그래서 학습 때 못 본 글자가 들어오면 Missing [UNK] token from the vocabulary 에러가 난다 (직접 확인함).

안전하게 쓰려면 이렇게

```
from tokenizers.trainers import WordPieceTrainer

tokenizer = Tokenizer(WordPiece(unk_token="[UNK]"))
trainer = WordPieceTrainer(vocab_size=8000,
                           special_tokens=["[UNK]"])
tokenizer.train(["../datasets/corpus.txt"], trainer=trainer)
```

- unk_token="[UNK]" : 모르는 글자를 [UNK] 로 처리
- WordPieceTrainer : 학습 설정을 따로 담는 객체
- special_tokens 에 [UNK] 를 넣어 사전에 등록
- vocab_size=8000 → BPE 실습과 맞춰서 비교가 공정해진다

EWHA EURON

#05 정규화 · 사전 토큰화 클래스 (표 5.6, 5.7)

표 5.6 정규화 (normalizers)

클래스	설명
NFD() / NFKD()	유니코드 분해 정규화 (K 는 호환 문자까지 통일)
NFC() / NFKC()	유니코드 결합 정규화
Lowercase()	모든 대문자를 소문자로
Strip()	문장 양 끝의 공백 제거
StripAccents()	모든 악센트 기호 제거 (é → e)
Replace()	문자나 정규표현식 기준으로 치환
BertNormalizer()	BERT 에서 쓴 정규화 그대로 적용
Sequence()	여러 정규화를 묶어 순서대로 실행

외우는 법 D = 분해 , C = 결합 ,
K = 호환 문자 (①, fi) 까지 표준 글자로

표 5.7 사전 토큰화 (pre_tokenizers)

클래스	설명
Sequence()	여러 사전 토큰화를 순서대로 실행
ByteLevel()	GPT-2 가 쓴 방법 . 바이트 단위 그래픽 문자로 매핑하고 공백 기준으로 나눔
CharDelimiterSplit()	지정한 문자 기준으로 나눔
Digits()	숫자 표현 기준으로 나눔
Punctuation()	구두점 기준으로 나눔
Metaspace(replacement="▁")	공백 기준으로 나누고 공백을 ▁로 바꿈 (SentencePiece 방식)
Whitespace()	단어 경계 (공백 · 구두점) 기준 . 정규식 \w+|[^\w\s]+ ← 이번 예제
WhitespaceSplit()	공백만 기준으로 나눔 (구두점은 안 나눔)
Split(pattern, behavior)	정규식과 동작 방식으로 나눔 . behavior : removed, isolated, merged_with_previous, merged_with_next, contiguous

비교 "Hello, world!"
Whitespace → Hello / , / world / ! WhitespaceSplit → Hello, / world!
\w+|[^\w\s]+ = 글자 · 숫자 덩어리 또는 기호 덩어리

EWHA EURON

#05 예제 5.16 — WordPiece 로 토큰화하기

코드

```
from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer = Tokenizer.from_file("../models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print(type(encoded_sentence))
print(encoded_sentence.tokens)
print([enc.tokens for enc in encoded_sentences])
print(encoded_sentence.ids)
print(tokenizer.decode(encoded_sentence.ids))
```

코드	의미
Tokenizer.from_file(...)	5.15 에서 저장한 JSON 불러오기
tokenizer.decoder = WordPieceDecoder()	④ 디코더 설정 (디코딩하려면 필요)
encode(sentence)	한 문장 → Encoding 객체
encode_batch(sentences)	여러 문장을 한 번에
type(...)	결과가 Encoding 객체인지 확인
.tokens	토큰 (문자열)
[enc.tokens for enc in ...]	여러 문장 결과에서 토큰만 꺼냄
.ids	정수 ID
decode(ids)	ID → 문장

출력 (교재 예시)

```
<class 'tokenizers.Encoding'>
['안녕하세요', '.', '워', '##드', '##피', '##스', '토', '##크', '##나이', '##저', '##입니다', '.']
```

EWHA EURON

#05 ## 의 의미와 메서드 비교

예제 5.16 단일 문장 토큰화 결과

안녕하세요	.	워	## 드	## 피	## 스	토	## 크	## 나이	## 저	## 입니다	.

- ## 가 붙은 토큰 = " 앞 토큰에 붙어 있는 이어지는 조각 " (토 + ## 크 + ## 나이 + ## 저 + ## 입니다 = 토크나이저입니다)
- WordPieceDecoder 는 ## 를 떼고 앞 토큰에 이어 붙여서 원래 단어로 복원한다
- SentencePiece 는 단어 시작에 ▁ , WordPiece 는 단어 이어짐에 ## → 표시하는 위치가 반대

작업	SentencePiece	허깅페이스 Tokenizers
모델 불러오기	SentencePieceProcessor().load("x.model")	Tokenizer.from_file("x.json")
한 문장 인코딩	encode_as_pieces / encode_as_ids 따로 호출	encode(문장) → Encoding 객체 하나에 다 들어 있음
여러 문장 인코딩	같은 메서드에 리스트를 넣음	encode_batch(리스트)
토큰 / ID 보기	반환값 자체	.tokens / .ids 속성
디코딩	decode_ids / decode_pieces	decode(ids) (디코더 설정 필요)
저장 형태	.model + .vocab 두 파일	.json 한 파일

*Encoding 객체 : 토큰화 결과 꾸러미 . tokens(토큰), ids(정수 ID) 외에 원문 위치 (offsets) 등이 들어 있다

EWHA EURON

#06 그 외 하위 단어 알고리즘

BBPE (Byte-level BPE)

유니코드 글자가 아니라 바이트 단위에서 BPE 를 수행한다
기본 단위가 바이트 256 개뿐이라 어떤 글자든 표현 가능 → <unk> 가 원천적으로 없다

사용처 GPT-2 이후 GPT 계열 (ByteLevel 사전 토큰화)

Unigram

BPE 와 반대 방향이다
크게 만든 어휘 사전에서 덜 필요한 토큰을 하나씩 제거하며 학습한다

사용처 SentencePiece 의 기본값 (model_type=unigram), T5, ALBERT 등

방향 정리

BPE · WordPiece	작은 조각에서 시작해 합쳐 나가는 상향식
Unigram	큰 사전에서 깎아 나가는 하향식

EWHA EURON

#06 총정리 비교표

	단어	글자	형태소	BPE	WordPiece
어휘 사전 크기	매우 큼	매우 작음	중간	직접 설정 (vocab_size)	직접 설정 (vocab_size)
OOV	많음	거의 없음	신조어에 약함	적음	적음
시퀀스 길이	짧음	매우 김	중간	중간	중간
학습 방식	–	–	사전과 규칙	데이터로 빈도 학습	데이터로 점수 학습
언어 지식 필요	띄어쓰기	없음	문법 지식 필요	필요 없음	필요 없음

결론 하위 단어 토큰화는 문법 지식 없이 데이터만으로 학습된다. 그래서 신조어나 오탈자가 생겨도 다시 학습하면 되고, 어떤 언어에도 적용할 수 있다. 사전 크기와 시퀀스 길이의 균형도 vocab_size 하나로 조절된다 → 현대 LLM 이 모두 하위 단어 토크나이저를 쓰는 이유

EWHA EURON

6. 임베딩



EWHA EURON

6.0 텍스트 벡터화

텍스트 벡터화 (Text Vectorization)

- 토큰화한 텍스트를 모델이 처리할 수 있는 숫자 벡터로 변환하는 과정
- 기초적인 텍스트 벡터화 종류: 원-핫 인코딩, 빈도 벡터화

1) 원-핫 인코딩 (One-Hot Encoding)

- 단어마다 고유 색인 부여 → 해당 색인 위치만 1로 표시하고 나머지 위치는 모두 0으로 표시

색인	0	1	2	3
토큰	I	like	apples	bananas

I like apples [1, 1, 1, 0]
I like bananas [1, 1, 0, 1]

EWHA EURON

6.0 텍스트 벡터화

2) 빈도 벡터화 (Count Vectorization)

- 문서에서 각 단어가 등장한 횟수를 기록하여 단어의 빈도를 벡터로 표현하는 방식
- 예: apples가 4번 등장 → 해당 위치에 벡터값 4 기록

원-핫 인코딩은 같은 단어가 여러 번 등장해도 1과 0으로 표현 → 빈도수 표현 불가능

기초 벡터화의 한계

- 벡터의 희소성 (Sparsity) ↑: 필요한 벡터 수에 비해 입력 문장 내 토큰 수가 적어 대부분의 값이 0
- 벡터는 입력 텍스트의 의미를 내포하지 X → 어순이 다르지만 의미적으로 유사한 문장의 벡터가 다르게 나타나거나, 어순이 다른 문장도 동일한 벡터로 표현될 수 있음

i scream for ice cream / ice cream for i scream [1, 1, 1, 1, 1]

EWHA EURON

6.0 텍스트 벡터화

3) 워드 임베딩 (Word Embedding)

- 기존 벡터화의 한계를 보완 – 단어의 의미를 학습해 표현
- 고정 길이로 표현된 실수 벡터에서 단어 의미를 다른 단어와의 상대적 위치로 표현

  → 단어 간의 관계 추론
- 고정 임베딩의 다의어 · 문맥 처리 한계는 동적 임베딩(Dynamic Embedding)으로 보완

EWHA EURON

6.1 언어 모델 (Language Model)

- 입력된 문장으로 각 문장을 생성할 수 있는 확률 계산
- 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측 수행
- 활용: 자동 번역, 음성 인식, 텍스트 요약

예시: 입력에 따른 후보 확률

입력	후보	교재 예시 확률
안녕하세요	만나서 반갑습니다.	0.081
안녕하세요	아니오. 괜찮습니다.	0.000001
대한민국 ___ 강남구	서울특별시	0.59
대한민국 ___ 강남구	아이스 아메리카노	0.000001

가능한 문장이 매우 다양하므로 문장 단위로 확률 계산하는 것은 어려움 → 토큰 단위로 하나씩 예측

EWHA EURON

6.1 언어 모델 – 1) 자기회귀 언어 모델

- 입력된 문장들의 조건부 확률을 계산하여 다음 단어를 예측하는 모델
- 이전에 등장한 모든 토큰을 이용해 문맥 파악하고 다음 토큰을 예측

언어 모델의 조건부 확률 – 다음 토큰의 조건부 확률

$$P(w_t \mid w_1, \ldots, w_{t-1}) = P(w_1, \ldots, w_t) \,/\, P(w_1, \ldots, w_{t-1})$$

조건부 확률의 연쇄법칙 적용 – 문장 전체 확률

$$P(w_1, \ldots, w_t) = P(w_1) \times P(w_2 \mid w_1) \times \cdots \times P(w_t \mid w_1, \ldots, w_{t-1})$$

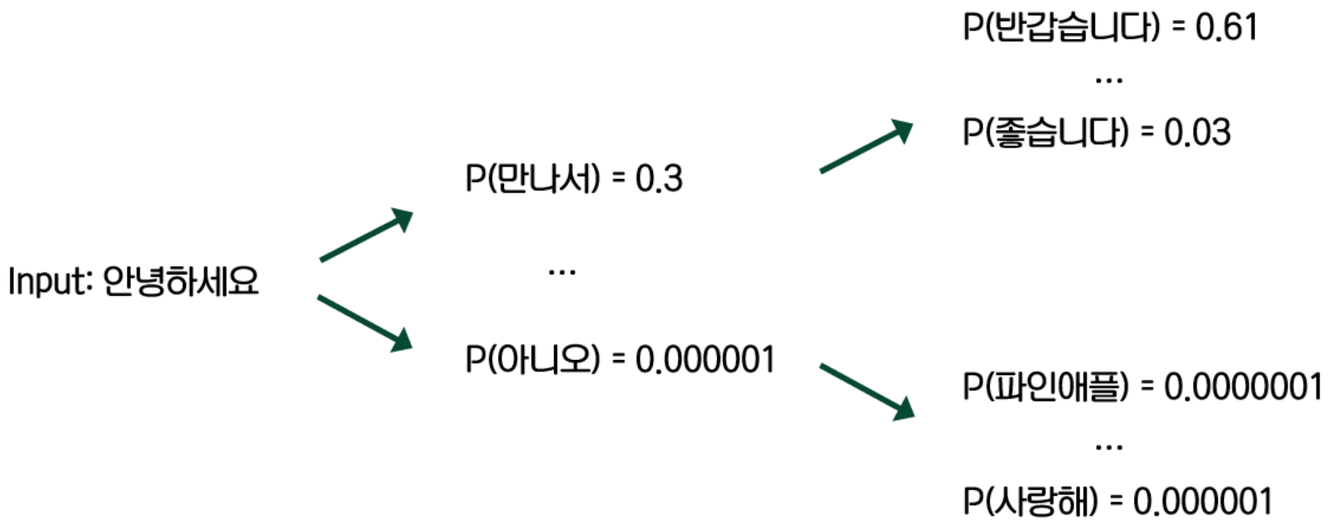
① 이전 토큰을 조건으로 다음 토큰의 확률을 계산

② 선택한 토큰을 입력 뒤에 붙이고, 다음 시점에서 같은 과정 반복

EWHA EURON

6.1 언어 모델 – 1) 자기회귀 언어 모델

예시



자기회귀 예측 과정

① 입력: 안녕하세요

P(만나서 | 안녕하세요) = 0.3

② 입력: 안녕하세요 만나서

P(반갑습니다 | 안녕하세요 만나서) = 0.61

...

∴ 이전에 등장한 모든 토큰의 정보를 활용해 입력된 문장의 문맥 정보 파악하고 다음 토큰 예측

매 시점마다 다음에 올 토큰을 예측하는 토큰 분류 문제!

EWHA EURON

6.1 언어 모델 – 2) 통계적 언어 모델

- 말뭉치에서 시퀀스의 확률 분포 추정 및 문맥을 파악 → 다음 등장할 단어의 확률을 예측
- 마르코프 체인: 이전과 현재 상태 간 전이 확률을 바탕으로 다음 상태 예측하는 빈도 기반 조건부 확률 모델

빈도 기반 조건부 확률

말뭉치의 시퀀스	등장 횟수	안녕하세요 다음에 올 확률
안녕하세요	1,000	기준 빈도
안녕하세요 만나서	700	700 / 1,000 = 0.7
안녕하세요 반갑습니다	100	100 / 1,000 = 0.1

통계적 언어 모델의 한계

- 단어의 순서와 빈도만을 기준으로 예측 → 문맥 제대로 파악 못 하면 부적절한 결과 초래 가능
- 데이터 희소성 문제: 관측한 적 없는 단어나 문장에 대한 확률 추정하기 어려움

EWHA EURON

6.1 언어 모델 – 3) 언어 모델의 사전 학습

GPT: 이어 쓰기

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT: 빈칸 채우기

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: BERT는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

최근 연구되는 자연어 처리 기법은 언어 모델을 활용해 가중치를 사전 학습함 (GPT, BERT)

EWHA EURON

6.2 N-gram

- N개의 토큰을 묶은 연속된 단어 시퀀스 단위로 분석하여, 특정 단어 시퀀스가 등장할 확률 추정

종류	N	분할 결과
Unigram	1	안녕하세요 / 만나서 / 진심으로 / 반가워요
Bigram	2	안녕하세요 만나서 / 만나서 진심으로 / 진심으로 반가워요
Trigram	3	안녕하세요 만나서 진심으로 / 만나서 진심으로 반가워요
N-gram	4↑	-

N-gram에서의 조건부 확률 – 이전 N-1개 토큰으로 다음 토큰 예측

$$P(w_t | w_{t-1}, w_{t-2}, \cdots, w_{t-N+1})$$

EWHA EURON

6.2 N-gram

N-gram 구현 방법 - Bigram

```
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

bigram = ngrams(sentence, 2) # ① Python
print(bigram)
bigram = nltk.ngrams(sentence.split(), 2) # ② NLTK
print(list(bigram))
```

출력 결과

```
[('안녕하세요', '만나서'), ('만나서', '진심으로'),
('진심으로', '반가워요')]
```

- N = 2 → words[0:]과 words[1:]을 짝지어 연속한 두 단어를 묶음
- 토큰 4개에서 2개씩 연속해 묶으면 3쌍
- 직접 구현과 NLTK 결과 같다

활용

- 연속 문자열 패턴과 관용적 표현 분석
- 순서가 중요한 자연어 처리 작업

EWHA EURON

6.3 TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency)

- 텍스트 문서에서 특정 단어의 중요도 계산하는 방법
- BoW에 가중치를 부여하는 방법

BoW (Bag-of-Words)

- 문서나 문장을 단어의 집합으로 표현하고 각 단어의 등장 횟수를 기록
- 등장 빈도 고려 but 단어별 중요도를 고려한 가중치는 X 모두 동일

문장	I	like	this	movie	don’t	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don’t like this movie	1	1	1	1	1	0	0

EWHA EURON

6.3 TF-IDF – 단어 빈도와 문서 빈도

1) 단어 빈도 (Term Frequency, TF)

$$TF(t, d) = count(t, d)$$

- 문서 내에서 특정 단어의 빈도수
- TF ↑ = 중요한 역할? < 전문 용어, 관용어
- 문서가 길면 단순 횟수도 커질 수 있음

2) 문서 빈도 (Document Frequency, DF)

$$DF(t, D) = count(t \in d : d \in D)$$

- 한 단어가 한 번 이상 등장한 문서의 수
- 한 문서에서 반복돼도 한 번만 집계
- DF ↑ = 일반적으로 널리 사용 = 중요도 ↓
- DF ↓ = 특정 문맥에서만 사용 = 중요도 ↑

movie	문서 1	문서 2	문서 3	전체
등장 횟수	2	1	1	4회
등장한 문서 수	1	1	1	3개

EWHA EURON

6.3 TF-IDF – 역문서 빈도와 TF-IDF

3) 역문서 빈도 (Inverse Document Frequency, IDF)

$$\mathrm{IDF}(t, D) = \log(\mathrm{count}(D) \,/\, (1 + \mathrm{DF}(t, D)))$$

- 문서 내에서 특정 단어가 얼마나 중요한지를 나타냄
- DF ↓ → IDF ↑
- 분모의 +1 : 0으로 나누는 경우를 방지 / 로그: 값의 범위를 줄임

TF-IDF

$$\mathrm{TF\text{-}IDF}(t, d, D) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t, D)$$

- 단어 빈도와 역문서 빈도를 곱한 값 = 중요도
- 한 문서에서 자주 나오면서 전체 문서에는 드물게 등장하는 단어에 높은 가중치

EWHA EURON

6.3 TF-IDF

Scikit-learn 라이브러리 설치

```
pip install scikit-learn
```

TF-IDF 클래스

```
tfidf_vector = sklearn.feature_extraction.text.TfidfVectorizer(
        input = "content",
        encoding = "utf-8",
        lowercase = True,
        stop_words = None,
        ngram_range = (1, 1),
        max_df = 1.0,
        min_df = 1,
        vocabulary = None,
        smooth_idf = True,
)
```

주요 매개변수

- input : 입력 데이터 형태
- encoding : 인코딩 지정
- lowercase : 소문자 변환 여부
- stop_words : 불용어. 분석에 도움이 되지 않는 의미 없는 단어. 사전에 추가 X
- ngram_range (최솟값, 최댓값)
- max_df / min_df : 문서 빈도가 너무 높거나 낮은 단어 제외
- vocabulary : 기존 단어 사전 사용 또는 학습 시 생성
- smooth_idf : 분모에 1을 더함

EWHA EURON

6.3 TF-IDF

TF-IDF 계산

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
        "That movie is famous movie",
        "I like that actor",
        "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```

주요 메서드

- fit(corpus) : 단어 사전과 IDF 학습
- transform(corpus) : 학습한 기준으로 문서 벡터화
- toarray() / vocabulary_ : 배열과 단어별 색인 확인

EWHA EURON

6.3 TF-IDF

열 색인	0 actor	1 don	2 famous	3 is	4 like	5 movie	6 that
문서 1	0	0	0.3969	0.3969	0	0.7937	0.2344
문서 2	0.6198	0	0	0	0.6198	0	0.4813
문서 3	0.4805	0.6317	0	0	0.4805	0	0.3731

문서별 상위 3개 단어·색인 순서

: [[2, 3, 5], [0, 4, 6], [0, 1, 4]]

단어 사전과 매핑

: [[famous, is, movie], [actor, like, that], [actor, don, like]]

∴ 문서마다 중요한 단어만 추출 가능, 벡터값 활용해 문서 내 핵심 단어 추출 가능

EWHA EURON

6.3 TF-IDF

빈도 기반 벡터화의 한계

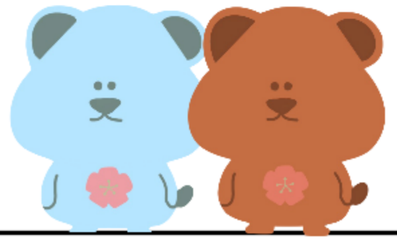
- 단어의 문서 내 중요도는 드러나지만, 순서와 문맥은 반영하기 어려움
- 문장 생성처럼 순서가 중요한 작업에는 부적합
- 벡터가 단어의 의미를 담고 있진 X

문서	교재 예문
1	나는 바나나를 먹었다
2	그녀가 과일을 섭취한다
3	고양이는 야옹 운다

- 1과 2는 의미상 서로 가깝지만, 세 문장은 서로 공유하는 단어가 없어 빈도 벡터로 계산한 세 문장 간의 유사도는 동일함

EWHA EURON

THANK YOU



EWHA EURON



10주차 질문

5장 토큰화 6장 임베딩

EWHA EURON

질문1: 한수연

Q. 어휘 사전을 키우면 문장당 토큰 수가 줄어 시퀀스가 짧아지지만, 말뭉치 크기가 고정되어 있으면 드문 토큰은 학습에서 거의 보지 못합니다. 이런 이유로 어휘 사전 크기를 늘려도 성능이 계속 좋아지지는 않는다면, 최적 크기는 말뭉치 크기와 언어(한국어와 영어)에 따라 어떻게 달라질까요?

EWHA EURON

질문2: 서영

Q. TF-IDF는 단어의 희소성을 중요도의 근거로 사용하는데, 이러한 가중치가 실제로 문서의 의미를 반영한다는 것을 어떻게 증명할 수 있을까?
또한 TF-IDF가 높은 가중치를 부여한 단어가 실제로 중요한 단어인지 판단할 객관적인 기준은 무엇일까? 또한 문서 집합의 구성이 달라지면 동일한 단어의 IDF 값도 달라지는데, 이를 단어의 고유한 중요도라고 해석하는 것이 타당할까?

EWHA EURON

논의: 안서영

Q. 논의: 교재에서는 형태소 분석이 단어의 의미와 문장 구조를 파악하는 데 유용하지만, 신조어나 외래어, 오탈자가 포함된 문장에서는 형태소 분석기가 단어를 잘못 분리할 수 있다는 한계를 소개했습니다. 예를 들어 '시보리'라는 단어를 '시'와 '보리'로 나누는 경우가 있습니다. 그렇다면 이러한 문제를 해결하기 위해 형태소 분석 대신 BPE나 WordPiece와 같은 하위 단어 토큰화를 사용하는 것이 더 효과적일까요? 두 토큰화 방식은 각각 어떤 장단점이 있으며, 어떤 상황에서 어떤 방식을 선택하는 것이 더 적절할까요?

EWHA EURON